



EE6008 Collaborative Research and Development Project

Project Charter

<<Project No. AY26S1-38-CME & Visual Object Detection Using Artificial Intelligence>>

Students' Names: <<PAN DENG, TAN SIMAI, CHEN SIYUAN, WU ZEFEI, CHEN JIONG>>

Supervisor's Name: << Yap Kim Hui >>

**School of Electrical and Electronic Engineering
Academic Year 2026/27
Semester 1**

Date Prepared:

Project No. & Project Title	AY26S1-38-CME & Visual Object Detection Using Artificial Intelligence
Project Supervisor	Yap Kim Hui
Team Leader	PAN DENG
Names of Team Members	TAN SIMAI, CHEN SIYUAN, WU ZEFEI, CHEN JIONG

Project Purpose or Justification:

- Mainstream deep-learning detectors such as the YOLO family are accurate but "closed-set": a model only recognizes the fixed categories it was trained on, and targeting a new object normally means collecting hundreds of labelled images and retraining. This makes conventional detection slow to adapt whenever an operator needs to find something the model has never seen — a specific tool on a workbench, one product on a shelf, a missing part, or an unusual object in a camera feed.
- Two further problems limit field usability. First, detection quality drops sharply under poor illumination (night-time, dim indoor, backlit scenes), which is exactly when automated monitoring matters most. Second, the manual image-labelling step that every custom detector depends on is tedious and is frequently the real bottleneck in a project schedule.
- This project tackles all three issues by building an interactive, prompt-driven detection and annotation tool. Instead of retraining, the user states the target through a text prompt (naming categories) or a visual prompt (uploading one example image and drawing a box around the object). A built-in low-light enhancement stage keeps detection reliable in dark conditions, and a semi-automatic annotation mode reuses the detector's own predictions to accelerate dataset creation. The outcome is a lightweight desktop aid that lets non-expert users apply object detection to new targets in minutes rather than days.

Project Description:

The team will develop a cross-platform desktop application (Python + PySide6) that combines multi-modal prompting, real-time detection, low-light enhancement and assisted annotation in one interface. The work has four parts.

1. Detection core. Pre-trained YOLOv8 models (nano / small / medium) will be exported to ONNX and executed through ONNX Runtime with selectable CPU or CUDA providers. A custom letterbox pre-processing, output-decoding and Non-Maximum-Suppression pipeline turns raw model output into stable boxes, with a user-adjustable confidence threshold and automatic fallback to a smaller model when a weight file is absent.
2. Multi-modal prompting. In text-prompt mode the user types one or more class names and detections are filtered accordingly. In visual-prompt mode the user uploads a reference image and drags a rectangle over the target on an interactive canvas; the cropped patch becomes a template, and each candidate region is scored against it by HSV colour-histogram correlation so that only visually similar instances are reported. We will also test a lightweight deep feature embedding to make the matching robust to colour-similar but shape-different objects.

Project Description:

3. Low-light enhancement. A Zero-DCE network will be integrated (also via ONNX) to brighten frames before detection, with a classical CLAHE-in-LAB algorithm as a no-model fallback. We will quantify how much this stage improves detection recall on dark footage.

4. Real-time pipeline and assisted annotation. A dedicated worker thread captures webcam or video input, applies enhancement and detection frame-by-frame, and streams annotated frames to the GUI without freezing it. An auto-annotation module then lets the user run the detector over an image folder and export accepted predictions as labels (YOLO txt / COCO json), turning the tool into a dataset-bootstrapping utility.

Deliverables: the working application, a benchmark report (accuracy and speed on CPU vs GPU, effect of low-light enhancement, visual-prompt matching quality) and user documentation.

Summary Milestones	Target Date
• Baseline System Setup & Detection Core Validation – Finalize the existing application framework; integrate YOLOv8 ONNX models and validate CPU/CUDA inference, preprocessing, NMS and real-time detection pipeline. Establish baseline accuracy and speed.	Week 1–2
• Multi-modal Prompting Development – Complete text-prompt and visual-prompt detection. Evaluate HSV histogram matching and develop/test a lightweight deep feature embedding approach for more robust visual matching.	Week 3–4
• Model Training, Parameter Optimization & Low-light Enhancement – Integrate and test Zero-DCE/CLAHE enhancement; conduct model training and optimize key parameters including confidence/IOU thresholds, input resolution and enhancement settings.	Week 5–7
• Assisted Annotation & Functional Integration – Develop the semi-automatic annotation module for image folders and support YOLO/COCO label export. Integrate detection, prompting, enhancement and annotation functions into the desktop application.	Week 8–9
• System Benchmarking & Further Optimization – Evaluate detection accuracy, precision/recall/mAP, visual-prompt matching quality and inference speed. Compare CPU vs GPU and normal vs low-light conditions; conduct ablation studies and further optimization.	Week 10–12
• Final Validation, Demonstration & Documentation – Complete system testing and final demonstration; summarize experimental results and prepare the benchmark report, user documentation and project presentation.	Week 13

	Team Member				
Activity in which a team member plays a key role	PAN DENG	CHEN JIONG	CHEN SIYUAN	TAN SIMAI	WU ZEFEI
Project management & timeline planning	A	I	C	R	I
Dataset preparation & annotation	R	R	I	A	C
Model design, training & optimization	R	C	A	R	I
System & function development	I	A	R	C	R
Performance evaluation, visualization & reporting	C	I	R	R	A

Summary Budget (if any):

The project runs almost entirely on open-source software (PySide6, OpenCV, ONNX Runtime, pre-trained YOLOv8 and Zero-DCE weights), so there is no software licensing cost. Anticipated expenses are minor and mostly optional. Testing will require one or two USB webcams or IP cameras for live-stream and low-light experiments, at roughly S\$30–80 in total, and these can otherwise be borrowed from the lab. Model export, fine-tuning and benchmarking will be carried out on the team members' personal machines or the School's existing GPU resources, at negligible cost, and test videos and model files will be shared through free-tier or existing institutional cloud storage. Overall the budget is negligible: no specialised hardware or paid datasets will be purchased, and all requirements can be met with equipment already available to the team or the School.

Risk Assessment (if lab equipment is used):

- No hazardous materials, high-voltage rigs or moving machinery are involved. The only physical equipment is a standard USB webcam or laptop camera, which poses no safety hazard and needs no special training. The main risks are technical and schedule-related.
- Visual-prompt reliability. Colour-histogram matching can confuse objects that share a colour palette but differ in shape. Mitigation: add shape / feature-based similarity, expose a tunable match threshold, and use text prompts where more appropriate.
- GPU and environment issues. CUDA / cuDNN / onnxruntime-gpu version mismatches on Windows can silently push inference back to CPU. Mitigation: detect and warn on CUDA fallback (implemented), document a tested environment, and keep the CPU path fully functional.
- Real-time performance. Enhancement plus per-frame detection may not sustain a usable frame rate on CPU-only machines. Mitigation: default to the nano model, allow frame skipping and lower input resolution, and profile the pipeline early.
- Low-light over-correction. Aggressive brightening can add noise or colour shift that harms detection. Mitigation: benchmark enhanced vs raw input and provide an on/off toggle (implemented).

Risk Assessment (if lab equipment is used):

- Schedule risk. The auto-annotation and advanced-matching features could absorb time needed for the core system. Mitigation: treat them as stretch goals, freeze the detection core by the mid-project milestone, and track weekly milestones with clear task ownership.